Programming Fundamentals-CS101

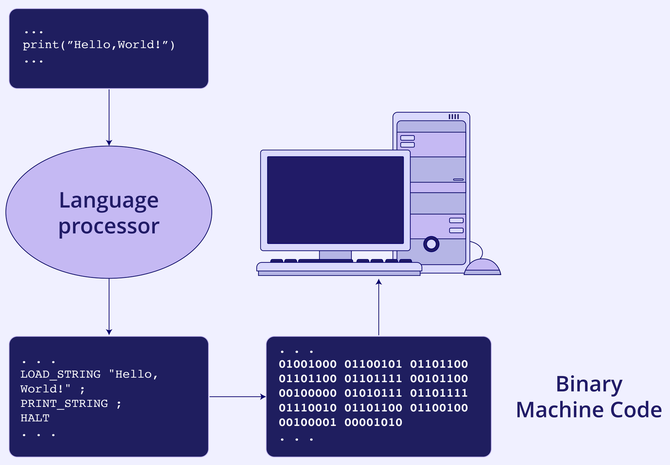
educative

What is Programming?

Programming is the process of giving instructions to a computer to perform a specific task.

Purpose

Programming solves problems and automates tasks.

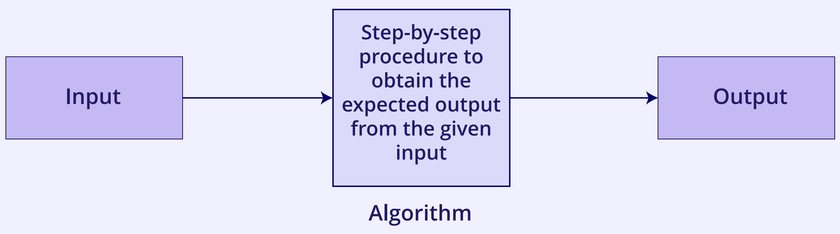


Algorithm

An algorithm is a step-by-step procedure for solving a problem.

Key points

- Algorithms are language-independent.
- Well-defined algorithms are essential for efficient programming.



Algorithm

Variables

Variables are storage locations with a symbolic name (identifier) and an associated value.

Purpose

Variables store and manipulate data in a program

Editor

```
// Example: Variable declaration
  age = 25
  name = "John"
```

Data Types

Data types are categories of data with predefined values and operations.

Common Types

Some examples include integers, floating-point numbers, characters, and booleans.

Editor

```
// Example: Common data types
  count = 10          // Integer
  price = 19.99       // Floating-point
  letter = 'A'        // Character
  isTrue = true       // Boolean
```

Operators

Operators are symbols that perform operations on variables and values.

Arithmetic Operators

They perform basic mathematical operations, i.e., addition **(+)**, subtraction **(-)**, multiplication **(*)**, and division **(/)**.

```
// Example: Arithmetic operations
a = 10
b = 5
result = a ⊙ b    // ⊙ could be any operator from this set {+, -, *, /}
```

Assignment Operators

They assign values to variables, i.e., assign **(=)**, add and assign **(+=)**, subtract and assign **(-=)**, multiply and assign **(*=)**, divide and assign **(/=)**.

Editor

```
// Example: Assignment operators
a = 10
a ⊙= 5    // ⊙ could be any operator from this set {+, -, *, /}
```

Comparison Operators

They compare values and return a boolean result **(true or false)**, i.e., equal to **(==)**, not equal to **(!=)**, greater than **(>)**, less than **(<)**.

```
// Example: Comparison operators
x = 8, y = 5
is_equal = (x == y)          // is_equal will hold false
is_not_equal = (x != y)      // is_not_equal will hold true
is_greater_than = (x > y)    // is_greater_than will hold true
is_less_than = (x < y)       // is_less_than will hold false
```

Logical Operators

They combine multiple conditions and return a boolean result, i.e., **and (&&)**, **or (||)**, **not (!)**.

Editor

```
// Example: Logical operators
x = 8, y = 5
var_1 = (x > 0 and y < 10)  // var_1 will hold true
var_2 = (x > 0 or y > 10)  // var_2 will hold true
var_3 = !(x == y)  // var_3 will hold true
```

Control Structures

They are constructs that manage the flow of a program.

Common Structures

Sequence

It executes statements in order.

Example: Making a sandwich

- Get bread.
- Apply peanut butter.
- Add jelly.
- Place another slice of bread on top.

Selection

It involves choosing between alternative paths in programming.

If-else

It is a conditional statement executing different codes based on a condition.

Editor

```
// Example: Deciding what to wear
weather = "Rainy"
if weather is "Sunny":
    // If it's sunny, wear comfortable clothes
    print("It's sunny! Wear shorts and a T-shirt.")
else:
    // If it's not sunny, wear something warm
    print("It's not sunny. Wear jeans and a sweater to stay cozy.")
```

Editor

```
// Example: Grading
marks = 85
if score >= 90:
    print("A grade - Excellent!")
else if marks < 90 and marks >= 80:
    print("B grade - Good job!")
else if marks < 80 and marks >= 70:
    print("C grade - Average performance.")
else if marks < 70 and marks >= 60:
print("D grade - Below average. Improvement needed.")
else:
    print("F agrade - Failed. Additional efforts required.")
```

Output

```
B grade - Good job!
```

Editor

```
// Example: Daily routine adviser
time_of_day = "afternoon"
weather = "sunny"
temperature = 25
day_of_week = "Friday"
if time_of_day == "morning":
    print("Good morning! Don't forget to have a hearty breakfast.")
if weather == "sunny":
    print("It's a sunny day! Don't forget your sunglasses.")
if temperature > 30:
    print("It's hot outside! Consider wearing sunscreen.")
if day_of_week != "Monday":
  print("We have a Discrete Math class!")
```

Output

```
It's a sunny day! Don't forget your sunglasses.
We have a Discrete Math class!
```

Iteration

It repeats a block of code (loops).

While loop

A while loop repeatedly executes a set of instructions as long as a specified condition remains true.

Programming Fundamentals-CS101

educative

Editor

```
// Example: WALL-E (AI Robot) cleaning up the room

set toys_count to 3

// While there are still toys to pick up
while toys_count is greater than 0:
    print("WALL-E picks up a toy.")
    decrement toys_count by 1

print("WALL-E has picked up all the toys.")
```

Output

```
WALL-E picks up a toy.
WALL-E picks up a toy.
WALL-E picks up a toy.
WALL-E has picked up all the toys.
```

For loop

A for loop iterates over a specified range or collection, executing a set of instructions for each element or iteration.

Editor

```
// Example: Greeting friends using the for loop

// List of friends
friends = ["Alice", "Bob", "Charlie", "David", "Eva"]

// For each friend in the list
for name in friends:
    print("Hello, ", name)
```

Output

```
Hello, Alice
Hello, Bob
Hello, Charlie
Hello, David
Hello, Eva
```

Do-while loop

A do-while loop repeatedly executes a set of instructions as long as a specified condition remains true, guaranteeing at least one iteration.

Editor

```
// Example: Catching butterflies
caught_butterflies = 0

repeat do:
    print("Chasing butterflies...")

    // Check if a butterfly is caught.
    print("Did you catch a butterfly? (yes/no): ")
    user_input = get_user_input()

    if user_input == "yes":
        increment caught_butterflies by 1
        print("Great job! You caught a butterfly.")
    else:
        print("Keep trying to catch a butterfly.")

// While the child has not caught any butterflies yet.
while caught_butterflies is 0
```

Output

```
Chasing butterflies...
Did you catch a butterfly? (yes/no): no
Keep trying to catch a butterfly.
Chasing butterflies...
Did you catch a butterfly? (yes/no): yes
Great job! You caught a butterfly.
```

Functions

Functions are named blocks of code that perform a specific task.

Purpose

Functions promote code reusability and organization.

Function definition and call

Editor

```
// Function definition
function_name(parameters):
    // Function body
    // ...
    return result
// Function call
result = function_name(arguments)
```

Editor

```
// Example: Addition function
add_numbers(num1, num2):
    sum = num1 + num2
    return sum

result = add_numbers(3, 5)

print("The sum is:", result)
```

Output

```
The sum is: 8
```

Input/Output

Input

The input obtains data from the user or external sources.

Output

The output displays the results to the user.

Editor

```
// Example: Input and output
  name = input("Enter your name: ")

  print("Hello, " + name + "!")
```

Output

```
Enter your name: John
Hello, John!
```

Comments

Comments are annotations in the code for human understanding, ignored by the computer.

Purpose

Comments improve code readability and explain complex sections.

Comment syntax in popular languages

- Use the // symbol for single-line comments in C++, Java, C#, and JavaScript, and # in Python.
- Use the `/*` symbol to open and the `*/` symbol to close multiline comments in C++, Java, C#, and JavaScript, and `""" """` symbols in Python.

Errors

Errors in programming are unexpected issues that disrupt the proper execution of a program.

Syntax Errors

These are mistakes in the structure or grammar of the code.

Editor

```
// Example 1: Misspelled keyword
print("Hello, world!")

// Example 2: Incorrect indentation
if x > 0:
print("Positive number")


// Example 3: Variable naming issue
2variable = 10

// Example 4: Unmatched parentheses
result = (5 + 2 * (3 - 1)

// Example 5: Undefined variable
print(undefined_variable)
```

Runtime Errors

These are errors that occur during execution (e.g., dividing by zero).

Editor

```
// Example: Division by zero
a = 10
b = 0
result = a / b
```

Logical Errors

In a logical error, the code runs without crashing but produces incorrect results.

Editor

```
// Example: Faulty Celsius to Fahrenheit Conversion

celsius_temperature = 20

// The formula mistakenly multiplies Celsius by 9/5 without adding 32
fahrenheit_temperature = celsius_temperature * 9 / 5
print("Fahrenheit temperature is ", fahrenheit_temperature) // Incorrect
```

Problem Breakdown Step-by-Step

Breaking down a problem into smaller subproblems and solving them step-by-step is a fundamental approach in programming. Here's a general guide on how to do it:

Problem Statement

The problem is to check if three given numbers (num1, num2, and num3) form a Pythagorean Triplet

1. Understand the problem

- The problem is to determine if the square of the largest number is equal to the sum of the squares of the other two numbers.
- Input: Three numbers, **`num1`**, **`num2`**, and **`num3`**.
- Output: **`True`** if they form a Pythagorean Triplet, **`False`** otherwise.

Programming Fundamentals-CS101

educative

2 Divide into subproblems

- Subproblem 1: Find the smallest number among the given three numbers.
- Subproblem 2: Find the largest number among the given three numbers.
- Subproblem 3: Calculate the square of the smallest number.
- Subproblem 4: Calculate the square of the middle number.
- Subproblem 5: Calculate the square of the largest number.
- Subproblem 6: Check if the sum of the middle and smallest squares equals the largest square.

3 Divide into subproblems

- **`min:`** Takes three numbers as input and returns the smallest one.
- **`max:`** Takes three numbers as input and returns the largest one.
- **`square:`** Takes a number as input and returns its square.
- **`is_pythagorean_triplet:`** Takes three numbers as input and returns **`True`** if they form a Pythagorean Triplet, **`False`** otherwise.

Editor

```
// Function to find the maximum of the three numbers
max(num1, num2, num3):
    if num1 >= num2 and num1 >= num3:
        return num1
    else if num2 >= num1 and num2 >= num3:
        return num2
    else:
        return num3

// Function to find the minimum of the three numbers
min(num1, num2, num3):
    if num1 <= num2 and num1 <= num3:
        return num1
    else if num2 <= num1 and num2 <= num3:
        return num2
    else:
        return num3

// Function to calculate the square of a number
square(number):
    return number ** 2


is_pythagorean_triplet(num1, num2, num3):
    // Find the largest, second largest, and third largest
   largest = max(num1, num2, num3)
   smallest = min(num1, num2, num3)
   middle = num1 + num2 + num3 - largest - smallest


return square(middle) + square(smallest) == square(largest)
```

4 Test the functions

- Test the **`min`** function with inputs 5, 4, 3.
  - Expected result: Minimum is 3
- Test the **`max`** function with inputs 5, 4, 3.
  - Expected result: Maximum is 5
- Test the **`square`** function with sample inputs.
  - Expected result: Square of 5 = 25
- Test the **`is_pythagorean_triple`**t function with sample inputs.
  - Expected result: **`True`**, as $3^2 + 4^2 = 5^2$ (5, 4, 3 is a Pythagorean Triplet)

Editor

```
// Test 1: min function
test1 = min(5)
print("Minimum is ", test1)

// Test 2: max function
test2 = max(5)
print("Maximum ", test2)

// Test 3: square function
test3 = square(5)
print("Square of 5 = ", test3)


// Test 4: is_pythagorean_triplet function
test4 = is_pythagorean_triplet(5, 4, 3)
if test4 == True:
    print("5, 4, 3 is a Pythagorean Triplet")
else:
    print("5, 4, 3 is not a Pythagorean Triplet")
```

5 Integrate functions

- Combine the results of the subproblems to get the final result.

6 Test the program

- Run the main program with specific inputs (**`num1 = 5, num2 = 4, num3 = 3`**).
- Expected output: **`"Forms a Pythagorean Triplet"`**

Editor

```
main_function(num1, num2, num3):
    result = is_pythagorean_triplet(num1, num2, num3)
    if result == True:
        print("Forms a Pythagorean Triplet")
    else:
        print("Does not form a Pythagorean Triplet")

// Running the main_function
main_function(5, 4, 3)
main_function(2, 4, 6)
```

Output

```
Forms a Pythagorean Triplet
Does not form a Pythagorean Triplet
```

7 Refactor and optimize

- In the given example or context, there is no explicit need to improve the code structure (refactoring) or execution efficiency (optimization).

8 Document the code

- Add comments to explain the purpose of the functions and complex sections.

Practice

Key to Mastery

Regular coding practice is essential for skill development.

Resources

- Utilize online platforms like Educative for structured learning.
- Engage in coding challenges to enhance problem-solving skills.
- Engage in real-world projects to apply your skills practically; Educative projects offer a hands-on experience, enriching your learning journey.

Note: We've kept things simple in this cheat sheet, using a universal style that fits any language. But as you explore different languages, remember each has its own syntax for writing code. Enjoy your coding journey!